EPFL
Faculty I&C

Master of Science in Computer Science

**Semester Project**
Winter Semester, 2006/07

**Asheesh Gulati**

---

# Creating installers for Java applications

---

supervisor

**David Portabella**

professor in charge

**Dr. Martin Rajman**

LIA
[Laboratoire d'Intelligence Artificielle]

# Table of Contents

# 1 Introduction

Installation (or setup) of a program is the act and the effect of putting a program in a computer system so that it can be executed.[1]

This process can be as simple as unpacking and copying on the local disk the files required by the application, or a lot more complex, taking into account customized settings and configuration of the computer in order to have the application operate correctly.

Because the requisite process varies for each application and each computer, many software come with an installer – a dedicated program which automates most of the work required for their installation.

## *1.1 Software installation*

The installation phase is the first contact the user has with the application, and it is very important that this first contact be as seamless and natural as possible.

There are two points every « good » installer should take care of:

1) the user should know at all time what is exactly going on, especially in case of unexpected situations

2) the user should always be in total control of the installation process

Thus the installer has to be, in some way, intelligent, as to allow the user to complete his objective, i.e. to have the application working on his computer, and to avoid unnecessary complicated steps preventing him to reach this goal.

This can become quite difficult if the software requires other applications to be already installed on the user's system. In this case, should the installer automatically try to install these third-party software or just ask the user to do so himself before continuing?

Another consideration is the platform on which the application is going to be installed. Generally, software are intended to be run on multiple platforms, creating the need to provide an installer for each of them.

Finally, the installer should display a language the user understands, thus the need to address the localization problem.

1 cf. http://en.wikipedia.org/wiki/Installer

## 1.2 Aim of this study

While big companies can make their own installer, or use a commercial solution to make one, there are lots of small projects composed of only few developers with no time to create an installer for their application or investigate available solutions. Very often, they simply do not provide an installer, which is a pity: this means that lots of users won't use their application, even if it is great, because it is not easy to install.

This project tries to present an overview of the solutions available to create installer programs for applications written in Java. As a concrete example, the ECatalog application will be used for illustrating the solutions revised in this study.

ECatalog is a database front-end that provides trade-off analysis to the user during the search. ECatalog can access Apache Derby (embedded and selected by default) and MySQL databases.

ECatalog has been developed at the Artificial Intelligence Laboratory at EPFL. More informations about the project are available on the SourceForge page http://ecatalog.sourceforge.net.

An installer for ECatalog must have the following features:

- check if a JRE meeting the version requirements is available on target system; if not, inform the user and (optionally) install one automatically
- install ECatalog on target system
- provide an uninstaller
- (optional) present MySQL as an alternative to Derby; if MySQL is not available on target system, install it automatically and configure ECatalog to use it
- (optional) on Windows platform, present Microsoft Access as an alternative to Derby; if the driver required to connect to Microsoft Access databases is not available on target system, install it automatically and configure ECatalog to use it
- (optional) provide a means to transparently update ECatalog when a new version is released

Target platforms are Microsoft Windows and Linux systems, and MacOs when possible.

A CD containing all source and build files for the solutions reviewed in section 4 of this study has been made available.

# 2 Platform-specific tools

## 2.1 Microsoft Windows Installer

The engine used for the installation, maintenance, and removal of software on modern Microsoft Windows systems, called Windows Installer, supports installation packages in MSI format. An MSI package contains the installation information, and often the program files themselves.

Windows Installer's features include a GUI framework, automatic generation of the uninstallation sequence and powerful deployment capabilities, and make it a viable alternative to stand-alone installer programs.

Quite a few commercial solutions can generate MSI installation packages as well as their own proprietary installer programs:

- Altiris Wise Installation (http://www.wise.com)
- Lindersoft SetupBuilder (http://www.lindersoft.com)
- Macrovision FLEXnet InstallShield & InstallAnywhere (http://www.macrovision.com)
- ScriptLogic Desktop Authority MSI Studio (http://www.scriptlogic.com)

## 2.2 Linux package management systems

A package management system is a collection of tools to automate the process of installing, upgrading, configuring, and removing software packages from a computer.[2]

Each Linux distribution has its own package management system:

- Advanced Packaging Tool (APT), used by Debian/GNU Linux and its derivatives
- Autopackage, a complementary package management system that can be installed on any distribution and intended to be used for managing non-core applications (http://autopackage.org)
- Portage and Paludis (http://paludis.pioto.org), used by Gentoo Linux
- RPM Package Manager, originally Red Hat Package Manager, developed by Red Hat for Red Hat Linux, now used by many Linux distributions (http://www.rpm.org)

Additionally, Alien (http://kitenet.net/~joey/code/alien.html) allows to convert a package format to

2 cf. http://en.wikipedia.org/wiki/Package_management_system

another package format.

### 2.3 MacOS X Installer

Included in with the operating system, Installer extracts and installs files out of package or metapackage[3] files (PKG format). Its purpose is to help developers create uniform software installers (using the PackageManager tool also included with the operating system).

A Software Delivery Guide describing software distribution using Installer is accessible at http://developer.apple.com/documentation/DeveloperTools/Conceptual/SoftwareDistribution/index.html.

# 3 Available solutions

Only free solutions are included here. Most of these are open-source projects.

## 3.1 Installers generators

### 3.1.1 Inno Setup

http://www.jrsoftware.org/isinfo.php

This free software is a tough opponent to many commercial solutions. It offers all the required features with one drawback: it can only create Windows executables.

### 3.1.2 InstallAnywhere Now

http://goldengate.zerog.com/releases/now

The company behind the InstallAnywhere suite, Zero G Software, has been recently bought by Macrovision. As a result, InstallAnywhere Now, the entry-level version of this solid, multi-platform commercial solution, has been discontinued.

Still, InstallAnywhere 5.5.1 Now can be found on the remnant of Zero G Software's website. Installer programs built with this “Unlicensed Trial Edition” display a notice stating that they were “created with a licensed but unregistered version of InstallAnywhere Now”

Delivering some nice features, including shortcuts, uninstaller, application launcher creation,

3 a package that contains one or more packages

InstallAnywhere 5.5.1 Now can also bundle a JVM with the installer.

### 3.1.3 InstallBuilder

http://www.bitrock.com/products_installbuilder_overview.html

Although a commercial solution, InstallBuilder can be used under a free license to create installers for open-source projects.[4] This multi-platform solution has all the standard features, can generate RPM packages (beta-version) and will soon be able to generate Debian packages.

### 3.1.4 IzPack

http://www.izforge.com/izpack

IzPack allows to create lightweight installers packaged in JAR archives that can be run on any system where a JVM is available. This is a nice open-source installer generator, if we don't bother about Java being already installed or not on the user's machine.

Other features comprise easy localization, flexible and powerful user interface, shortcuts creation system and automatic uninstaller creation.

http://maven-plugins.sourceforge.net/maven-izpack-plugin

The Maven IzPack Plug-in allows to create a Windows executable off an IzPack installer archive with an optionally bundled JVM.

### 3.1.5 Nullsoft Scriptable Install System

http://nsis.sourceforge.net/Main_Page

NSIS is an open source software that allows to create Windows installers. It was developed by Team Nullsoft to easily distribute Winamp, their now famous media player.

Full-packed with features, small and flexible, NSIS is script-based and easy to use. Unfortunately, it is limited to Windows platforms.

### 3.1.6 Spoon Installer

http://sourceforge.net/projects/spoon-installer

A nice installer generator exclusively for Windows.

4 cf. http://www.bitrock.com/products_installbuilder_opensource.html

### 3.1.7 VAInstall

http://vainstall.sourceforge.net

VAInstall is a SourceForge project that has most of the required features (localization, shortcuts and uninstaller creation). Nevertheless, it lacks the ability to bundle a JVM with the installer.

Like NSIS, the installer generation is driven by scripts.

### 3.1.8 WiX

http://wix.sourceforge.net

The Windows Installer XML (WiX) toolset, released on SourceForge by Microsoft under the Common Public License, builds Windows Installer (MSI) packages from an XML document.

WiX seems to be the only open-source option for developers who want to produce installation packages in MSI format.

## *3.2 Executable wrappers*

### 3.2.1 JSmooth

http://jsmooth.sourceforge.net

Another SourceForge project, JSmooth is an executable wrapper that creates Windows executables from JAR files. Its major feature is the feedback it provides to the user if a JVM is not found on target system, helping him downloading and installing one.

### 3.2.2 Launch4j

http://launch4j.sourceforge.net

Launch4j is also an executable wrapper that creates Windows executables from JAR files. Additionally, it allows to create launchers for JAR or class files without wrapping.

# 4 Explored approaches

Three main approaches were explored: application launchers, installer programs and Java Deployment. For the first two approaches, a subset of the solutions presented in the previous section was explored, which contains mainly promising open-source projects that have a high activity percentile The reader will find more open-source and commercial solutions reviewed in the comparison table in section 5.

As each solution gives examples using ECatalog, here is the layout of the application:

```
ecatalog
  |
  |- build (contains the compiled classes)
  |
  |- configs (contains example configuration files)
  |
  |- databases
  |    |
  |    `- derby (contains derby databases specified in example configuration files)
  |
  |- ext (contains external libraries in JAR format)
  |
  |- images (contains the images used by the interface)
  |    |
  |    `- features (contains images specified in one of the example configuration files)
  |
  |- logs (contains log files created by the application each time it is executed)
  |
  `- src (contains the source code)
```

All folders need to be copied on user's machine, except `src` and `logs`.

The main method is located in ecatalog.ECatalog.

## *4.1 Application launchers*

The easiest solution for a developer to distribute an application may be to provide an executable or a script to run it on some operating system. Even though there is no installation process, this simple approach for merits further consideration.

### 4.1.1 Java Archive

This file format is a container for Java classes and related resources, and is intended to store an

application in order to easily distribute it. A manifest file, located inside the archive at `/META-INF/MANIFEST.MF`[5], describes how the JAR will be used. It is possible to directly execute an application packaged in a JAR provided that the manifest file specifies the "Main-Class", i.e. the class acting as entry point to the application (`MANIFEST.MF` for ECatalog):

```
Manifest-Version: 1.0
Class-Path: . ext/activation.jar ext/bsf_core.jar ext/bsf_debug.jar
ext/derby.jar ext/dpc.jar ext/jaxb-api.jar ext/jaxb-impl.jar ext/js.jar
ext/jsr173_api.jar ext/mysql-connector-java-3.1.12-bin.jar ext/orbital-
ext.jar ext/serializer.jar ext/TableLayout.jar ext/xalan.jar
ext/xercesImpl.jar ext/xws-security.jar
Main-Class: ecatalog.ECatalog
```

The "Class-Path" refers to files outside of the JAR file, which can be confusing and inconvenient when distributing applications making use of external libraries also packaged in JAR files: they have to be provided together with the JAR containing the application and can't be packaged inside.

Indeed, the Java classloader sun.misc.Launcher$AppClassLoader, which takes over at the start of the command used to execute JAR files (`java -jar`), only knows how to do two things:

- load classes/resources that appear at the root of a JAR file
- load classes/resources that are in codebases pointed to by the `META-INF/MANIFEST.MF` Class-Path attribute

Moreover, it deliberately ignores any environment variable settings for CLASSPATH or the command-line argument `-cp` that you supply. And it does not know how to load classes or resources from a JAR file that is contained inside another JAR file.

This clearly limits the possibility to have one single file for distribution. One option would be to unjar all the class files from external libraries and jar them together with the application class files, but this is only applicable if the license protecting the external library explicitly allows repackaging. If this is not the case, One-JAR would be the other option of choice (cf. next section).

On the other hand, it is possible to load resources (such as configuration files, images) contained inside a JAR file using the getResource mechanism (cf. section 4.3.1 to see how this works).

The following command creates an executable Java Archive with the application class files only:

```
jar cvmf MANIFEST.MF ecatalog.jar -C build .
```

Obviously, a Java Runtime Environment is needed to run an application packaged in a JAR. The following command executes a Java Archive:

```
java -jar ecatalog.jar
```

Usually with graphical systems, the user can simply double-click on a JAR file to execute it. This

5 The first "/" designates the root of the JAR.

constitutes one of the main reasons to use this solution.

An executable JAR for ECatalog contains only the compiled classes and all other resources (configs, databases, ext and images) must be provided with it. For example, the whole can be compressed and distributed as a ZIP file.

Executable JAR files for ECatalog are located under `Application_Launchers/Java_Archive` on the accompanying CD.

Reference:

- Lesson: Packaging Programs in JAR Files (Java tutorial on deployment), http://java.sun.com/docs/books/tutorial/deployment/jar/index.html

### 4.1.2 Multi-platform launcher using One-JAR

One-JAR allows to easily package an application in an executable JAR along with the external libraries (in JAR format) it requires. This is interesting when the developer wants to distribute an application as one single executable file, but can't repackage the libraries because of license issues.

One-JAR needs the developer to use the following layout:

```
|
|- main (contains the main application as an executable JAR)
|
`- lib (contains the libraries in JAR format)
```

And a manifest file specifying the main class and the class path (`MANIFEST.MF` for ECatalog):

```
Manifest-Version: 1.0
Class-Path: . build/ main/ecatalog.jar lib/activation.jar lib/bsf_core.jar
lib/bsf_debug.jar lib/derby.jar lib/derbyclient.jar lib/dpc.jar lib/jaxb-
api.jar lib/jaxb-impl.jar lib/js.jar lib/jsr173_api.jar lib/mysql-connector-
java-3.1.12-bin.jar lib/orbital-ext.jar lib/serializer.jar
lib/TableLayout.jar lib/xalan.jar lib/xercesImpl.jar lib/xws-security.jar
Main-Class: ECatalogApplication
```

(The main application in this case is the ecatalog.jar generated in the previous section.)

These two folders and their contents can then be archived using the following command:

```
jar cvmf MANIFEST.MF ecatalog_.jar main lib
```

The final step is to update the resulting executable JAR with One-JAR bootstrap environment: decompress One-JAR (`one-jar-boot.jar`) in a sub-folder (called `boot` in the example), enter the sub-folder, and update the application JAR file.

```
mkdir boot
```

```
cd boot
jar -xvf ../one-jar-boot.jar
jar -uvfm ../ecatalog_.jar boot-manifest.mf .
```

That's it, the application is now packaged in one single executable.

One-JAR boostrapped executable JAR file for ECatalog is located under `Application_Launchers/One-JAR` on the accompanying CD.

References:

- One-JAR project page, http://one-jar.sourceforge.net
- Simplify your application delivery with One-JAR, http://www-128.ibm.com/developerworks/java/library/j-onejar/

### 4.1.3 Windows launcher using Launch4j

Now, there is a great probability that a normal Windows user does not know what to do with an executable JAR file, but all normal Windows users know about standard Windows EXE files.

Thanks to Launch4j, it is possible to wrap a JAR file into a Windows executable, i.e. to produce a Windows executable that contains the JAR file and that can launch the application. Options, e.g. the icon to use for the executable, or the Java version required by the application, can be specified in the configuration file (`ecatalog-config.xml` excerpt):

```
<launch4jConfig>
  <headerType>0</headerType>
  <outfile>ecatalog.exe</outfile>
  <jar>ecatalog.jar</jar>
  <icon>app.ico</icon>
  <classPath>
    <mainClass>ecatalog.ECatalog</mainClass>
    <cp>build/</cp>
    <cp>ext/activation.jar</cp>
    <cp>ext/bsf_core.jar</cp>
    <cp>ext/bsf_debug.jar</cp>
    <cp>ext/derby.jar</cp>
    <cp>ext/dpc.jar</cp>
    <cp>ext/jaxb-api.jar</cp>
    <cp>ext/jaxb-impl.jar</cp>
    <cp>ext/js.jar</cp>
    <cp>ext/jsr173_api.jar</cp>
    <cp>ext/mysql-connector-java-3.1.12-bin.jar</cp>
    <cp>ext/orbital-ext.jar</cp>
    <cp>ext/serializer.jar</cp>
    <cp>ext/TableLayout.jar</cp>
    <cp>ext/xalan.jar</cp>
    <cp>ext/xercesImpl.jar</cp>
    <cp>ext/xws-security.jar</cp>
  </classPath>
  <jre>
```

```
    <minVersion>1.5.0</minVersion>
    <maxVersion>1.5.0_11</maxVersion>
  </jre>
  <versionInfo>
    <fileVersion>1.0.0.0</fileVersion>
    <txtFileVersion>1.0</txtFileVersion>
    <fileDescription>database frontend with tradeoff analysis</fileDescription>
    <copyright>EPFL-LIA</copyright>
    <productVersion>1.0.0.0</productVersion>
    <txtProductVersion>1.0</txtProductVersion>
    <productName>ECatalog</productName>
    <internalName>ecatalog</internalName>
    <originalFilename>ecatalog.exe</originalFilename>
  </versionInfo>
</launch4jConfig>
```

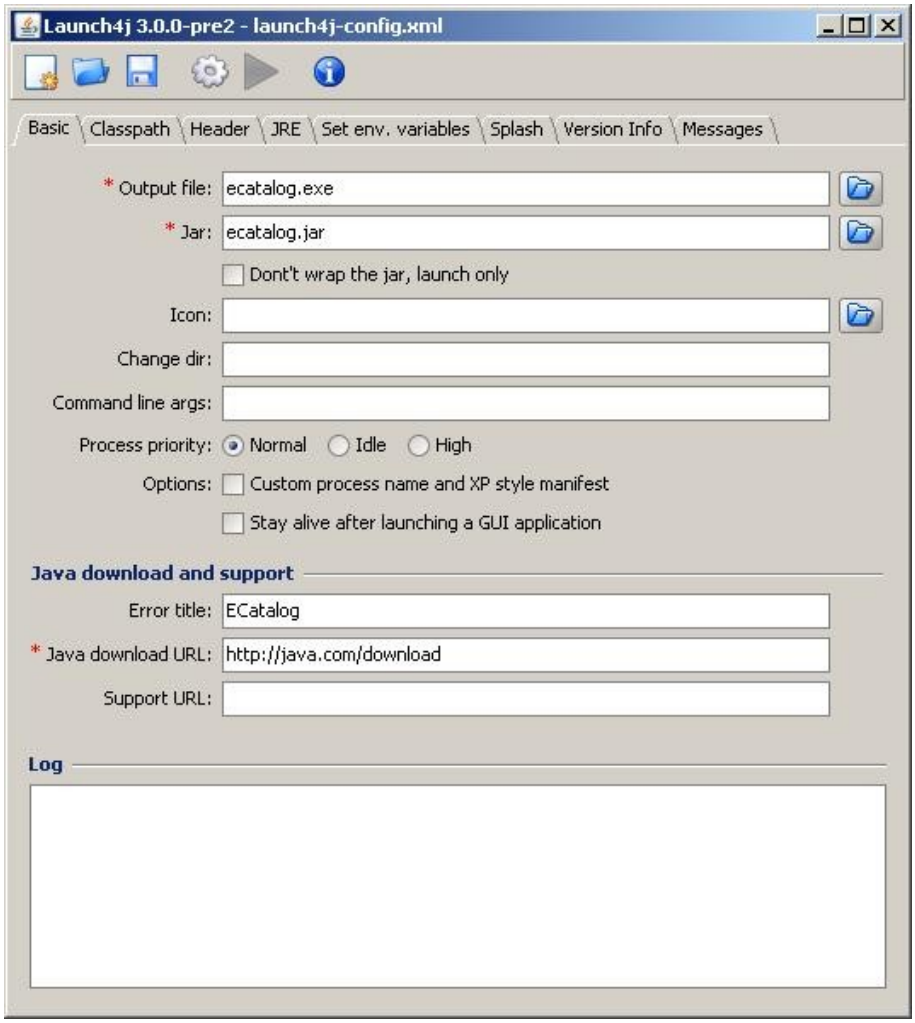
`headerType` specifies which Java command to use to launch the application: java or javaw, designed for graphical programs.
`outfile` specifies the name of the executable to generate.

It is possible to create a Windows executable without wrapping the application JAR file by using the `<dontWrapJar>` tag with value `true`.

The following command creates the executable on Windows:

```
launch4jc ecatalog-config.xml
```

The command `launch4j` without argument enters the GUI mode. On Linux, there is only one shell script named `launch4j` that can be called with or without command line argument.




If a Java Runtime Environment is not found, the launcher warns the user and starts the default web browser to download one. The download URL can be specified in the configuration file[6] if different from http://www.java.com.

And, last but not least, it is also possible to specify a bundled JRE that the launcher can use to run the application (`ecatalog-config.xml` excerpt):

```
<jre>
  <path>jre1.5.0_11</path>
  <minVersion>1.5.0</minVersion>
  <maxVersion>1.5.0_11</maxVersion>
</jre>
```

If for some reason the bundled JRE can not be found, the launcher adopts the same behavior as before. This is very useful, because there is no need to generate two different executables, one that uses a bundled JRE and one that does not; it is sufficient to provide or not the bundled JRE.

As with the Java Archive solution, the application has to be compressed to be easily distributed.

6 It seems that this option has been removed in the last version of Launch4J.

Launch4j-generated Windows executables are located under `Application_Launchers/Launch4j` on the accompanying CD. Two zip files are available: one with a bundled JRE and one without.

Reference:

- Launch4j project forums, http://sourceforge.net/forum/?group_id=95944

## 4.2 Installer programs

An installer program is an executable for a specific platform that sets up an application on the user's computer. The desired features are:

- check if a Java Runtime Environment meeting the version requirements is available on target system; if not, inform the user and either install one or use a bundled one
- install ECatalog on target system
- provide an uninstaller

The first feature requires ECatalog to be bundled either with a JRE or the installer application to install one during setup. This raises a number of issues:

1) One has to notice that the size of the JRE gets bigger and bigger with every new version. As a comparison, JRE 1.1 is 2.64 MB and JRE 1.5 is 15.82 MB[7] (cf. table 4.1 for details). It is therefore not possible anymore to provide an application bundled with a JRE that still keeps a reasonable size. (**Update:** Java 6 seems to change this tendency for the installer size.)

| **Java Release (for Windows)** | **installer size** | **size on disk** | **download URL** | **description** |
|---|---|---|---|---|
| | **(International)** | | | |
| JRE 6 | 13.16 MB | 75.6 MB | link | link |
| JRE 5.0 | 15.82 MB | 69.5 MB | link | link |
| J2RE 1.4 | 15.24 MB | 40.3 MB | link | link |
| | **(English/International)** | | | |
| J2RE 1.3 | 5.26 MB / 7.94 MB | 17.3 MB / 22.8 MB | link | link |
| J2RE 1.2 | 5.14 MB / 7.18 MB | 15.3 MB / 19.8 MB | link | bug fixes |
| JRE 1.1 | 2.64 MB / 5.22 MB | ? | link | bug fixes |

2) Usually, Java applications have to be provided with a version of the JRE that the user has to install himself. This is because there are no easy means to install a JRE during the installation process of another application. As a result, the user has to go through two installation phases before being able to use the application.

7 JRE offline installers for Microsoft Windows; JRE 1.1 is US English and JRE 1.5 is multi-language

3) Even if a silent install of the JRE is used to address the previous issue, a series of dependency problems could happen, for example if other versions of the JRE are already installed, and if some application needs a specific version in order to run.

Another problem is related to Linux distributions: the version of Java available by default is limited to 1.4. In fact, Java 5.0 could not be included with Linux distributions until recently because of a licensing problem[8]. Even installing it manually was a pain, depending on the distribution.

This leads us to prefer a method that is smarter: if a JRE is not found on the user's computer, the installer program has to give precise feedback to help him download and install one (cf. JSmooth).

### 4.2.1 Installers for Windows, Linux and MacOs using InstallAnywhere Now

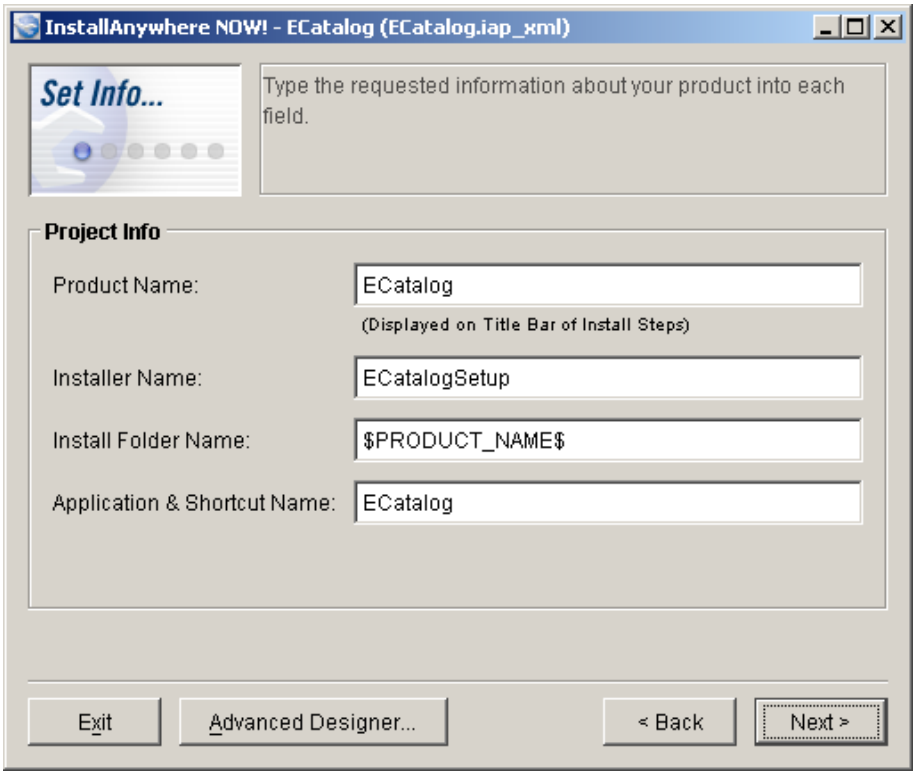
InstallAnywhere[9] was described in section 3.1.2. In this section, a practical step-by-step example is provided.

InstallAnywhere starts in "Wizard" mode, where all the options for the generation are specified through a nice user interface following a step-by-step procedure, but it is possible to switch to the "Advanced Designer" mode, displaying a full view of the project, at any time.

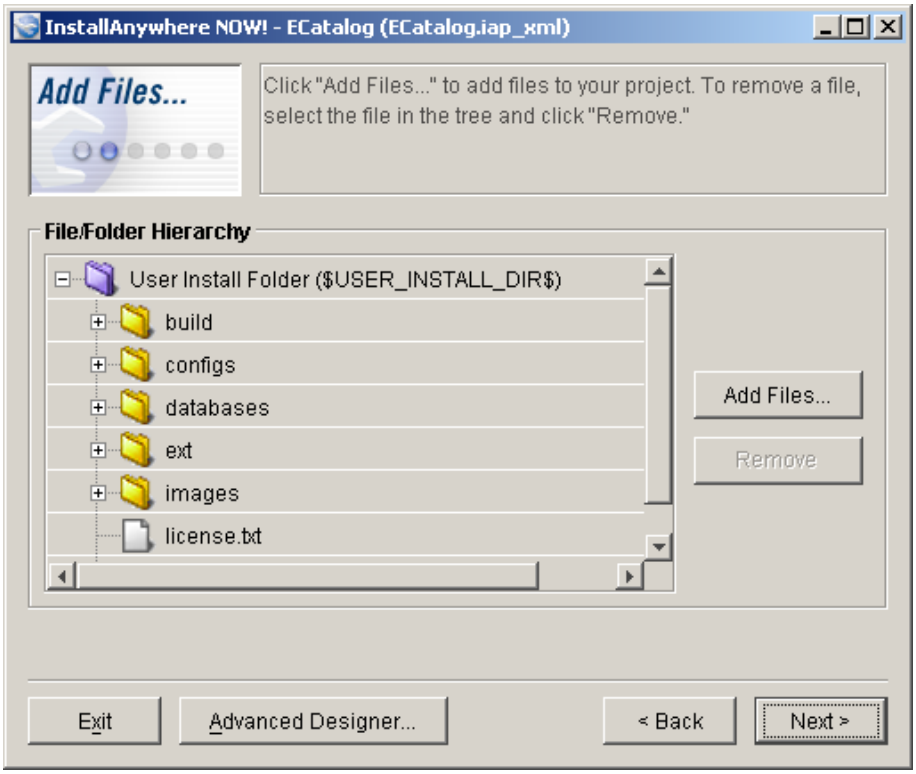
1. The first step is to create a project and to provide informations such as the application name, installer name or the name of the installation directory.

8 The new license, the Operating System Distributor License for Java (DLJ), is more permissive than the Binary Code License (BCL): cf. http://www.sun.com/smi/Press/sunflash/2006-05/sunflash.20060516.4.xml.

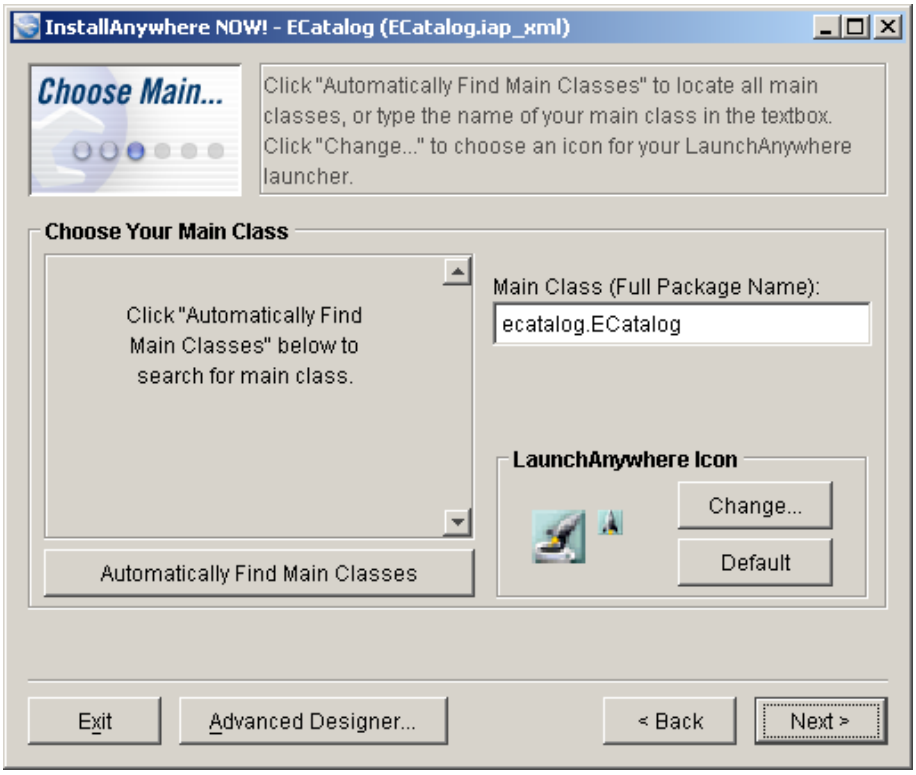
9 InstallAnywhere 5.5.1 Now can be downloaded at http://goldengate.zerog.com/releases/now.



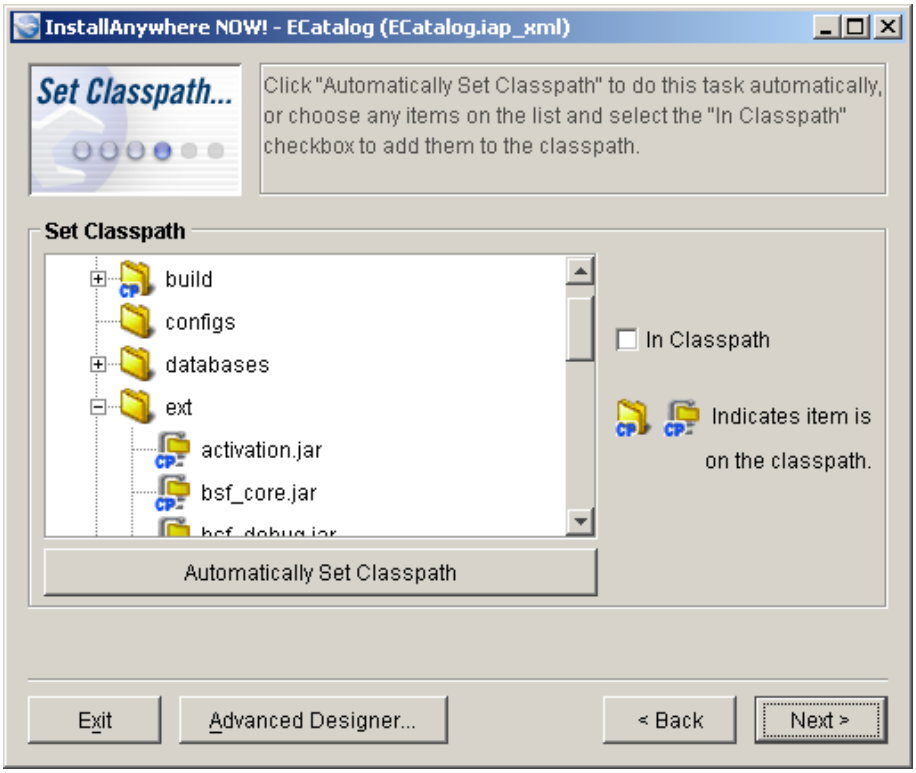

2. The second step is to add files to the project and specify the layout of the installation directory.



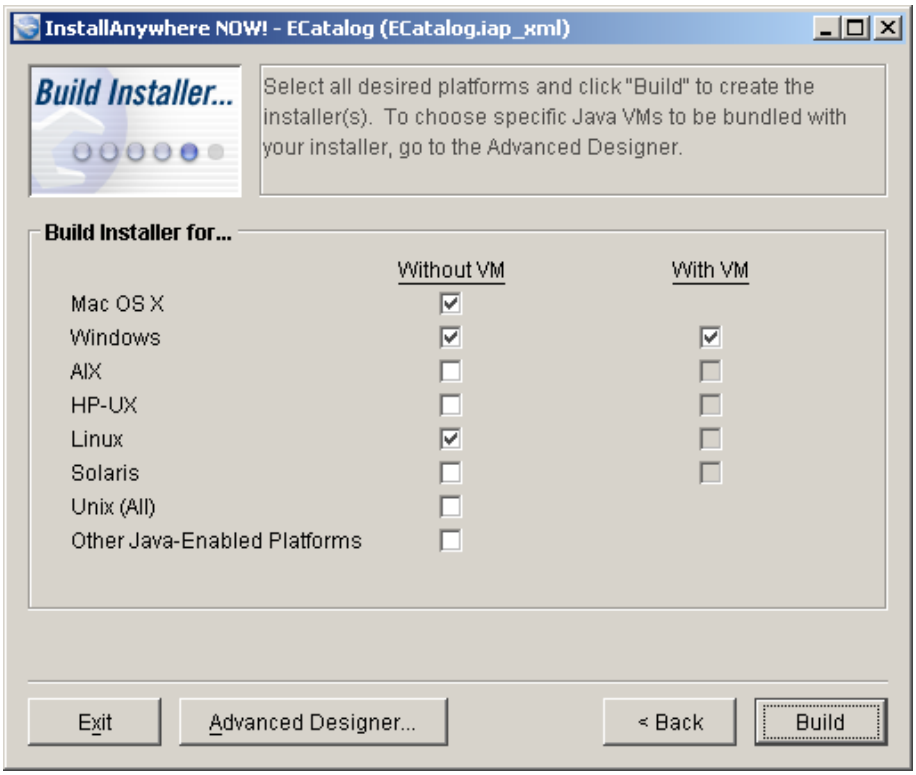
3. The third step is to indicate the Main Class for the application and the icon to use for the shortcuts.




4. The fourth step is to set the classpath for the application.

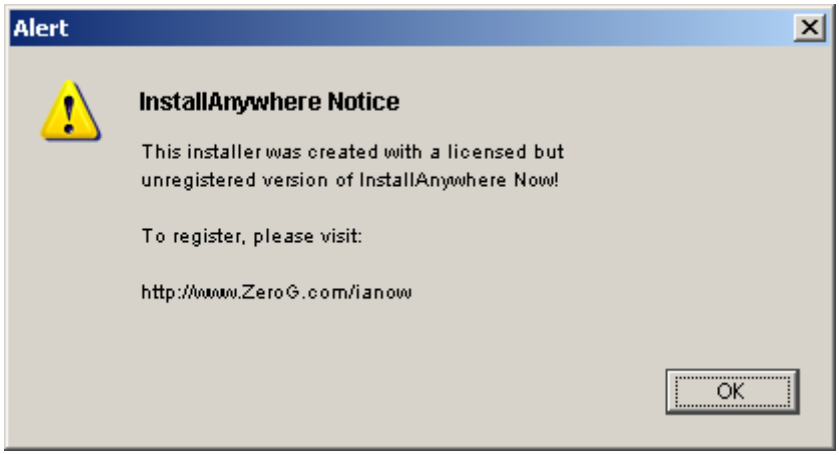


5. And the last step is to select the target platforms and generate the installer programs.




In this last step, it is possible for some platforms to specify if the installer must bundle a Java Virtual Machine to execute the application or not. (In the picture, a disabled "With VM" box means that the VM for this platform has to be downloaded[10].) A `jre` folder for the Java VM will be created at the root of the installation directory.

As this is a trial version of InstallAnywhere, generated installers display the following notice:




The installers can check if a Java Runtime Environment is available on target system, and its version. If a JRE is not found, or if the version doesn't match, the installer warns the user and

10 VM Packs for different Java versions are available at http://www.macrovision.com/downloads/products/flexnet_installshield/installanywhere/java_vm.shtml.

terminates. The installer also creates a launcher for the application at the root of the installation directory with shortcuts to the desktop and the programs menu. The application is also added to Windows registry, thus adding it to the "Add or Remove Programs" list in the Control Panel for easy uninstallation.

From this trial version, it is easy to imagine the possibilities of the latest full commercial version of InstallAnywhere.

InstallAnywhere Now-generated installers are located under `Installer_Programs/InstallAnywhere_Now` on the accompanying CD. Four versions are available: two Windows installers, one with a bundled JRE, one without, a Linux installer in binary format and a Mac OS installer.

### 4.2.2 Windows and Linux installers using VAInstall

VAInstall requires two files: the main configuration and the filelist.

The main configuration file specifies all the options needed in the installer generation, such as the filelist to use, the destination platform, the Java versions supported, etc. (`ecatalog-config.vai` excerpt):

```
# Name of installer
vainstall.archive.installClassName=ECatalogSetup

# Path of file listing files to include in the archive
vainstall.archive.filelist=ecatalog_filelist.txt

# JVM version
# the lower version supported by the application
vainstall.java.version.min=1.5

# targets: java, jar, jnlp, unix, linux-i386, win95 (comma separated)
vainstall.destination.targets=jar,win95,unix
```

The last variable is important, because it specifies the output form or the target platform for the installer: `java` for a Java class, `jar` and `jnlp` for an executable JAR, `unix` for an executable shell script on Unix/Linux, `linux-i386` for a native Linux-i386/glibc2 executable (`unix` target should be preferred to this one), and `win95` for Windows. An installer will be created for each specified target.

The filelist indicates all the files to package with the installer. These files will be copied to the target system following the specified structure (`ecatalog_filelist.txt` excerpt):

```
# (Flags)<OriginBase>|<DestBase>|<CommonPath>[|<*.ext>,!<*.ext>,...]

/workspace/ecatalog/configs|configs|
/workspace/ecatalog/databases|databases|
/workspace/ecatalog/ext|ext|
/workspace/ecatalog/images|images|
```

```
/workspace/ecatalog/logs|logs|!*.xml
/workspace/ecatalog/ecatalog.jar|ecatalog.jar|
/workspace/ecatalog/|license.txt|license.txt|
/workspace/ecatalog/readme.html|readme.html|
```

It is also possible to specify the creation of a launcher for the application (`ecatalog_filelist.txt` excerpt):

```
# {
#   <ScriptType>
#   <KeyWord1>=<arg1>
#   <KeyWord2>=<arg2>
#   ...
# }

{
  JarLauncher
  ScriptName=ecatalog
  Jar=ecatalog.jar
  JavaMode=windows
  CreateShortcutOnDesktop=true
}
```

`<ScriptType>` can be `JavaLauncher` (the launcher will execute a defined class with the Java VM) or `JarLauncher` (the launcher will execute a defined jar with the Java VM).

The installer generation can then be started with the following command (`vainstall.jar` must be in the classpath):

```
java com.memoire.vainstall.VAInstall ecatalog.vai
```

The installers can check if a Java Runtime Environment is available on target system, and its version. If a JVM is not found, or if the version doesn't match, the installer displays a warning and terminates.

An uninstaller script is generated at the root of the installation directory and an uninstaller JAR file is created under `C:\Program Files\Common Files\vainstall` on Windows, and `/usr/share/vainstall` on Linux.

VAInstall is a nice solution with the only weak point of being too “talkative”: too many unnecessary panels and confirmation boxes are presented to the user.

VAInstall-generated installers are located under `Installer_Programs/VAInstall` on the accompanying CD. Three versions are available: a Windows installer, a Linux installer in shell script format, and an executable JAR.

### 4.2.3 Multi-platform installer in JAR format using IzPack

IzPack generates a JAR file containing an installer and the resources to install based on an XML

configuration file (ecatalog.xml, excerpt):

```
<?xml version="1.0" encoding="iso-8859-1" standalone="yes" ?>

<installation version="1.0">

    <info>
        <appname>ECatalog</appname>
        <appversion>1.0</appversion>
        <authors>
            <author name="David Portabella Clotet"
                    email="david@portabella.name"/>
        </authors>
        <url>http://ecatalog.sourceforge.net</url>
        <javaversion>1.5</javaversion>
    </info>

    <guiprefs width="640" height="480" resizable="no"/>

    <locale>
        <langpack iso3="eng"/>
    </locale>

    <resources>
        <res id="LicencePanel.licence" src="license.txt"/>
        <res id="InfoPanel.info" src="readme.txt"/>
        <res id="shortcutSpec.xml" src="izpack-shortcutSpec.xml"/>
        <res id="Unix_shortcutSpec.xml" src="izpack-Unix_shortcutSpec.xml"/>
    </resources>

    <panels>
        <panel classname="HelloPanel"/>
        <panel classname="InfoPanel"/>
        <panel classname="LicencePanel"/>
        <panel classname="TargetPanel"/>
        <panel classname="PacksPanel"/>
        <panel classname="InstallPanel"/>
        <panel classname="ShortcutPanel"/>
        <panel classname="SimpleFinishPanel"/>
    </panels>

    <packs>
        <pack name="Base" required="yes">
            <description>Core files</description>
            <!-- this list specifies all the files or folders that need to be
                 copied during the installation -->
            <file src="build" targetdir="$INSTALL_PATH"/>
            <file src="configs" targetdir="$INSTALL_PATH"/>
            <file src="databases/derby" targetdir="$INSTALL_PATH/databases"/>
            <file src="ext" targetdir="$INSTALL_PATH"/>
            <file src="images" targetdir="$INSTALL_PATH"/>
            <file src="logs" targetdir="$INSTALL_PATH"/>
            <file src="app.ico" targetdir="$INSTALL_PATH"/>
            <file src="license.txt" targetdir="$INSTALL_PATH"/>
            <file src="readme.txt" targetdir="$INSTALL_PATH"/>
            <file src="ecatalog.jar" targetdir="$INSTALL_PATH"/>
            <!-- the following files will be copied during installation only
                 on target os (ecatalog.exe and ecatalog.sh are launchers for
```

```
                ECatalog) -->
            <file src="ecatalog.exe" targetdir="$INSTALL_PATH" os="windows"/>
            <file src="ecatalog.sh" targetdir="$INSTALL_PATH" os="unix"/>
            <!-- the following tag is used to execute a script after
                 installation; here it is used to set the executable attribute
                 on the linux launcher (the launcher itself is not executed, as
                 stage equals never) -->
            <executable targetfile="$INSTALL_PATH/ecatalog.sh"
                        stage="never"
                        os="unix"/>
        </pack>
        <pack name="Sources" required="no">
            <description>The sources</description>
            <file src="src" targetdir="$INSTALL_PATH"/>
        </pack>
    </packs>

    <native type="izpack" name="ShellLink.dll"/>

</installation>
```

The configuration is quite simple and flexible, and it is possible to choose the dialogs to display under panels. If more than one language is specified under locale, a dialog will let the user select one of them (unfortunately, this dialog appears too “cryptic”).

Remark: under Windows , the uninstall script is not added to the registry and the application is thus not listed in “Add or Remove Programs”.

The following command creates the installer:

```
compile ecatalog.xml -o EcatalogSetup.jar
```

Java Archives can only be executed on systems where a JRE is available. To overcome this limitation, IzPack-generated installers can use a native launcher that will first check target system for a Java Runtime Environment. The idea is to provide an executable for target system that acts as a launcher for the JAR. Two versions are available: v1.3 (based on wxWidgets) and v2.2 (based on QT). Of course, other executable wrappers can be used alternatively to IzPack native launcher, e.g Launch4j or Jsmooth.

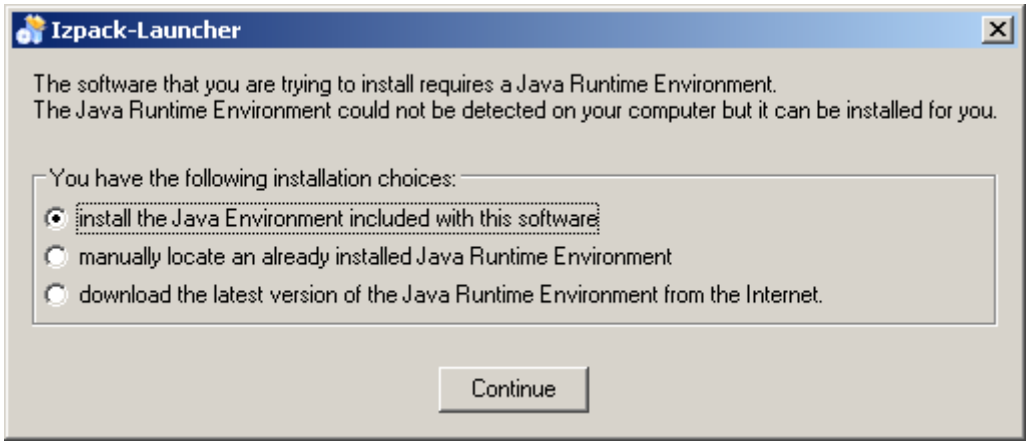
The IzPack native launcher is configured through a simple initialization file (`launcher.ini`):

```
# Global entries, can be overriden by specific ones.
jar = ECatalogSetup.jar
download = http://sun.java.com/

# Win32 specific entries
[win32]
jre = jre/jre-1_5_0_11-windows-i586-p.exe
```

Specific entries for other platforms can be added, but will be relevant only depending on the native launcher used, as the native launcher is itself an executable for a given operating system. These entries specify the path to the JRE installer bundled with the application.

If a JRE is found, then it will launch the installer, if not, then it will display three options to the user:




If for some reason the bundled JRE can not be found, the native launcher will still display the other two options.

The native launcher adds several files and folders to the application JAR, so the whole has to be compressed for easy distribution. The only problem of this solution is the size of the executable if a JRE installer was bundled with the installer application (26.2 MB).

IzPack-generated installers are located under `Installer_Programs/IzPack` on the accompanying CD. Three versions are available: an installer in JAR format and two Windows installers using IzPack native launcher (with and without a bundled JRE).

### 4.2.4 Windows installer using IzPack + 7-Zip

A Windows executable can be created using the previous (IzPack-generated) installer and 7-Zip SFX (self-extracting executable) feature. The archive can be set to automatically run the native launcher upon decompression (`config.txt` excerpt):

```
;!@Install@!UTF-8!
Title="ECatalog 1.0"
RunProgram="launcher-Win32.exe"
;!@InstallEnd@!
```

The first step is to compress the IzPack installer along with the native launcher files using 7-Zip (the resulting archive is called `files.7z` in the example), then to create the SFX by concatenating 7zS.sfx (from 7-Zip installation folder), the configuration file and the archive file:

```
C:\> copy /B 7zS.sfx + config.txt + files.7z EcatalogSetup.exe
```

(`copy /B a + b + c d` concatenates `a`, `b` and `c` in a file named `d`.)

The main advantage of this solution is that the user only sees a Windows executable. The installation starts as soon as the archive finishes auto-extracting the files in the default temporary

folder, which is nice compared to the previous solution, because the user does not need to extract anything himself. The disadvantage is the limitation to Windows platform. The file sizes are almost similar as with the previous solution.

IzPack + 7-Zip-generated installers are located under `Installer_Programs/IzPack_7-Zip` on the accompanying CD. Two versions are available: one with and one without a bundled JRE.

References:

- Building Native Windows Installers with IzPack Native Launcher, http://www.javalobby.org/articles/izpack/

## *4.3 Java Deployment*

The solutions seen so far are all off-line approaches: the user has to execute a program to install the desired software on his machine. No connection to the Internet is needed during the process, except maybe to download a JRE meeting the version requirements.

Now, if the hypothesis is made that a Java Runtime Environment is already available on target machine, two on-line approaches can be investigated: Java applet, and Web Start application. The added benefit of Java Deployment is that an application doesn't need installers for each specific platform. The desired features are:

- run ECatalog on target system
- transparently update ECatalog when a new version is released

Additionally, it seems possible to detect Java from a web page using a JavaScript. The user can then be redirected to an information page if Java is not installed or enabled in the browser. Unfortunately, it appears that a different piece of code is required for every browser, and sometimes for different versions of a same browser. The general idea is to initialize a mock applet in the web page and check the returned object in the script. If the reference is null, then the applet could not be created because Java is not installed/enabled. More informations are available in the following thread on the Sun Developers Forums :

- ANSWER: HOW TO DETECT Java Plugin from JavaScript, http://forum.java.sun.com/thread.jspa?threadID=168544

An alternate technique specific to applet deployment is to use a different HTML tag depending on the web browser and control through dynamic versioning the Java version requirements and the messages or redirections if Java is not available. Here, again, JavaScript can do the trick to detect the user's browser and generate the correct tag. The following page from the Java Plugin Guide explains the details of this technique :

- Using applet, object and embed Tags, http://java.sun.com/j2se/1.5.0/docs/guide/plugin/developer_guide/using_tags.html

### 4.3.1 Web Start

Java Web Start allows the user to automatically download and launch full-featured applications with a single click from a web browser. The current version is 1.2 and is shipped as part of Sun J2SE 1.4.1 or higher.

As with Java Applet (cf. next section), Web Start applications are, by default, run in a restricted environment with limited access to local computing resources. In this sandbox environment, Java Web Start can guarantee that a downloaded and potentially untrusted application cannot compromise the security of the local files or the network. It is thus mandatory to sign all JAR files containing code that will access local data. This allows the user to know that an application has not been modified since it was signed. This way, an application can request additional system privileges if the user decides to trust the certificate of the source.

The following practical considerations must be kept in mind when developing applications for deployment with Java Web Start:

- An application must be delivered as a set of JAR files.
- All application resources, such as files and images, must be stored in JAR files and they must be referenced using the getResource mechanism in the Java 2 platform (see below).
- If an application is written to run in a secure sandbox, it must follow these restrictions:
  - No access to local disk.
  - All JAR files must be downloaded from the same host.
  - Network connections are enabled only to the host from which the JAR files are downloaded.
  - No security manager can be installed.
  - No native libraries may be used.
  - Limited access to system properties. The application has read/write access to all properties defined in the JNLP file (`<property>` tag, described below), as well as read-only access to the same set of properties that an applet has access to[11].
- An application is allowed to use the System.exit call.
- An application that needs unrestricted access to the system will need to be delivered in a set of signed JAR files. All entries in each JAR file must be signed.

Java Web Start is based on the Java Network Launching Protocol and API (JNLP)[12], which defines, among other things, a standard file format that describes how to launch an application (`ecatalog.jnlp` excerpt):

```
<?xml version="1.0" encoding="utf-8"?>
<jnlp spec="1.0+"
    codebase="http://www.the-claw.net/ecatalog"
```

11 See http://lopica.sourceforge.net/ref.html#sandbox-props for a list.

12 the JNLP specification was developed via the Java Community Process (JCP), cf. http://jcp.org/en/jsr/detail?id=56

```
    href="ecatalog.jnlp">

<information>
    <title>ECatalog</title>
    <vendor>EPFL-LIA</vendor>
    <homepage href="http://ecatalog.sourceforge.com"/>
    <description>ECatalog</description>
    <description kind="short">
       ECatalog is a database front-end which provides trade-off analysis to
       the user during the search.
    </description>
    <offline-allowed/>
</information>

<resources>
    <j2se href="http://java.sun.com/products/autodl/j2se"
       onclick="javascript:mytracker(this.href);" version="1.5+"/>
    <jar href="core/ecatalog.jar"/>
    <jar href="ext/ecatalog-apartments.jar"/>
    <extension href="ecatalog-ext1.jnlp"/>
    <extension href="ecatalog-ext2.jnlp"/>
    <extension href="ecatalog-ext3.jnlp"/>
</resources>

<application-desc main-class="ECatalogLauncher"/>

</jnlp>
```

This example specifies 3 other JNLP files as extensions to the core ECatalog application. It is not possible to list all the extensions in one single JNLP file due to security restrictions which will be presented later.

The application cannot use disk-relative references to retrieve resources, as Java Web Start determines where to store the JAR files on the local machine. All application resources must be either retrieved from the JAR files specified in the resources section of the JNLP file or requested from the the Internet.

System properties available to the application can be defined under resources:

```
<resources>
    ...
    <property name="database" value="apartments" />
</resources>
```

The following code shows how the getResource mechanism works (`ECatalog.java` excerpt):

```
java.net.URL configFile = ECatalog.class.getResource(configFileName);
config = (ECatalogConfigType)((JAXBElement<?>)u.unmarshal(configFile)).getValue();
```

Path to resources loaded this way must be specified using slashes (/) as separators and must start with a slash (in the above example, `configFileName` is set to "`/configs/config-apartments.xml`").

The following setting is used to request full access to a client system, provided that all the JAR files not included as extensions are signed (`ecatalog.jnlp` excerpt):

```
<security>
   <all-permissions/>
</security>
```

Java Web Start presents a security warning displaying the details of the certificate. The user can also decide to always trust the source by adding the certificate to an "Approved certificates" list.

The implementation of code signing in Java Web Start is based on the security API in the core Java 2 Platform. Code signing is done using public-key cryptography. Code signed using the private key of the signer can be run on client machines once the public key corresponding to the signer is deemed as trusted on the respective machine. In order to inspire confidence in the user, developers associate their public key with a certificate, signed by a trustworthy certification authority (e.g. VeriSign, Comodo or thawte), as a way to prove their identity . This way, the message displayed to warn the user appears less intimidating.

A self-signed test certificate can be created and used to sign code by using the standard keytool and jarsigner tools from the JDK:

```
C:\>keytool -genkey
Tapez le mot de passe du Keystore :  pwd4keystore
Quels sont vos pr0nom et nom ?
  [Unknown] :  Asheesh Gulati
Quel est le nom de votre unit0 organisationnelle ?
  [Unknown] :  LIA
Quelle est le nom de votre organisation ?
  [Unknown] :  EPFL
Quel est le nom de votre ville de r0sidence ?
  [Unknown] :  Lausanne
Quel est le nom de votre 0tat ou province ?
  [Unknown] :
Quel est le code de pays α deux lettres pour cette unit0 ?
  [Unknown] :  CH
Est-ce CN=Asheesh Gulati, OU=LIA, O=EPFL, L=Lausanne, ST=Unknown, C=CH ?
  [non] :  oui

Sp0cifiez le mot de passe de la cl0 pour <mykey>
       (appuyez sur Entr0e s'il s'agit du mot de passe du Keystore) :  pwd4mykey

C:\>keytool -list
Tapez le mot de passe du Keystore :  pwd4keystore

Type Keystore : jks
Fournisseur Keystore : SUN

Votre Keystore contient 1 entr0e(s)

mykey, 20 f0vr. 2007, keyEntry,
Empreinte du certificat (MD5) : 2B:0E:C4:C2:86:55:2A:3F:F4:9E:13:49:E6:F9:A8:F0

C:\>jarsigner ecatalog.jar mykey
Enter Passphrase for keystore: pwd4keystore
Enter key password for mykey: pwd4mykey
```

```
Warning: The signer certificate will expire within six months.
```

A self-signed test-certificate should not be used to release production code. Instead, a certificate from a certification authority should be acquired. It is possible to request a free certificate from a certification authority such as thawte[13]. A certificate from a well-established trusted source will still be displayed to the user: in the end, the decision to run or not the code is in his hands.

There are two rules to remember with code signing:

- all JAR files that perform privileged operations must be signed
- all JAR files mentioned in one JNLP file must be signed with the same certificate

This is why the main JNLP file for ECatalog (`ecatalog.jnlp`) refers to three others JNLP files: `ecatalog-ext1.jnlp` specifies all non-signed JAR files, `ecatalog-ext2.jnlp` specifies a JAR file that was originally signed by the vendor (Sun in the example) and `ecatalog-ext3.jnlp` specifies JAR files signed by the author.

Finally, parameters can be passed to the application using an `argument` tag (`ecatalog.jnlp` excerpt):

```
<application-desc main-class="ecatalog.ECatalog">
    <argument>apartments</argument>
</application-desc>
```

These parameters are accessed just like other command line arguments, using the String args[] array passed to the main function.

Alternatively, it is also possible to set a system property using, in the resources tag (`ecatalog.jnlp` excerpt):

```
    <parameter name="database" value="apartments"/>
```

The property that can then be simply loaded from the code (`ECatalog.java` excerpt):

```
String configFileName = "/configs/config-" + System.getProperty("database") + ".xml";
```

The last step is to link the JNLP file in a web page. The first time a user requests a JNLP file from a web page, the web browser may ask for which application to use to open the file. If Java was properly installed on the user's machine, then Web Start will be proposed by default. The user can then choose to associate JNLP files with Java Web Start.

The Java Web Start version of ECatalog is located under `Java_Deployment/Web_Start` on the accompanying CD.

References:

13 cf. http://www.dallaway.com/acad/webstart/index.html for a step by step procedure

- Java Web Start Guide, http://java.sun.com/j2se/1.5.0/docs/guide/javaws/developersguide/contents.html

### 4.3.2 Applet

Java applets exist since the beginnings of Java as a programming language and are one of the reasons of its popularity. Applets are applications written to be embedded inside a web page and run in the context of a browser enabled with Java technology. Plug-ins for Internet Explorer and Mozilla/Netscape web browsers can be optionally installed when a JRE is installed.

For a developer, the following practical considerations must be taken into account when writing (or converting an existing application to) an applet, additionally to the considerations discussed in the previous section:

- The main class of the applet must be a subclass of java.applet.Applet, or of its subclass javax.swing.JApplet if it uses Swing components to construct its GUI.

- The web browser manages an applet life cycle by calling certain methods.
  - init: This method is intended for whatever initialization is needed for the applet. It is called after the `param` arguments of the `applet` tag. This method is equivalent to the main method of a stand-alone application and is thus mandatory.
  - start: This method is automatically called after *init* method. It is also called whenever user returns to the page containing the applet after visiting other pages.
  - stop: This method is automatically called whenever the user moves away from the page containing applets. This method can be used to stop an animation.
  - destroy: This method is only called when the browser shuts down normally.

Similarly to Web Start applications, applets are run in a secure sandbox and the code needs to be digitally signed to be able to request access to the local machine.

In the case of ECatalog, as most of the required modifications were already done in the Web Start application, the conversion to an applet was rather straightforward (`ECatalog.java` excerpt):

```
public void init() { // replaces main()
  try {
    String database = getParameter("database");
    this.configure("/configs/config-" + database + ".xml");
    //ecatalog.setLog(new ECatalogLog(ecatalog, getLogFileName()));

    if (log != null) log.start();

    db = new Database();
    db.init(config, log);

    db.updateContext();

    gui = new MainGui();

    gui.init(this);
    update();
```

```
    if (log != null) log.genericInfo("ready");

    UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
    this.getContentPane().add(this.getComponent());
  } catch (Exception e) {
    e.printStackTrace();
  }
}
```

In order to load an applet in a web page, the applet main class must be specified within the `applet` tag of the HTML page (parameters can be provided with a `param` tag):

```
<applet
   code     = "ecatalog.ECatalog"
   width    = "1200"
   height   = "900"
   codebase = "."
   archive  = "core/ecatalog_applet.jar, ext/ecatalog-apartments.jar,
ext/activation.jar, ext/bsf_core.jar, ext/bsf_debug.jar, ext/derby.jar,
ext/derbyclient.jar, ext/dpc.jar, ext/jaxb-api.jar, ext/jaxb-impl.jar,
ext/js.jar, ext/jsr173_api.jar, ext/mysql-connector-java-3.1.12-bin.jar,
ext/orbital-ext.jar, ext/serializer.jar, ext/TableLayout.jar, ext/xalan.jar,
ext/xercesImpl.jar, ext/xws-security.jar, databases/testdb.jar">

   <param name="database" value="apartments" />

</applet>
```

It is to be noted that the `applet` tag is deprecated as of HTML 4.0 and replaced by the `object` tag. But the official Java tutorial on deployment states: “However, the specification is vague about how browsers should implement the object tag to support Java applets, and browser support is currently inconsistent. It is therefore recommended that you continue to use the applet tag as a consistent way to deploy Java applets across browsers on all platforms.”

With the introduction of Java Web Start, the use of applets to deploy complete applications over the web is less appealing. The main difference between these two approaches is that an applet runs in the context of a web browser. Thus, applets are particularly well suited for applications that aim to provide more interactivity to web pages.

The applet version of ECatalog is located under `Java_Deployment/Applet` on the accompanying CD.

References:

- Java Plugin Guide, http://java.sun.com/j2se/1.5.0/docs/guide/plugin/developer_guide/contents.html
- Lesson: Applets (Java tutorial on deployment), http://java.sun.com/docs/books/tutorial/deployment/applet/index.html

# 5 Comparison table

| installers generator | generates uninstaller? | JRE detection? (bundles JRE?) | multi-language? | creates shortcuts? | dialog customization? | target platforms | notes |
|---|---|---|---|---|---|---|---|
| Desktop Authority MSI Studio | yes | / | ? | yes | yes | Windows | commercial product; generates only MSI packages |
| FLEXnet InstallShield | yes | ? | yes | yes | yes | Windows | commercial product |
| Inno Setup | yes | ? | yes | yes | yes | Windows | free product |
| InstallAnywhere | yes | yes (yes) | yes | yes | yes | Windows, Mac OS, Linux, Unix platforms | commercial product |
| InstallAnywhere Now | yes | yes (yes) | yes | yes | yes | Windows, Mac OS, Linux, Unix platforms | unlicensed free version; discontinued |
| InstallBuilder | yes | yes (?) | yes | yes | yes | Windows, Mac OS, Linux, Unix platforms | commercial product; supports RPM generation and integration |
| IzPack | yes | yes (yes) | yes | yes | yes | / | open-source product; generates only JAR; uninstaller not linked in Windows “Add/Remove programs” |
| Nullsoft Scriptable Install System | yes | ? | yes | yes | yes | Windows | open-source product |
| SetupBuilder | yes | ? | yes | yes | ? | Windows | commercial product |
| Spoon Installer | ? | ? | ? | ? | ? | Windows | open-source product |
| VAInstall | yes | yes (no) | yes | yes | ? | Windows, Linux, Unix platforms | open-source product |
| Wise Installation | yes | ? | yes | yes | yes | Windows | commercial product |
| WiX | yes | / | ? | yes | ? | Windows | open-source product; generates only MSI packages |

# 6 Conclusion

*"Building an installer is the last thing the developer does but the first thing the customer sees."*[14]

It is of critical importance for a software to be easy to install, but very often developers do not have enough time or energy to work on an installer program for their applications. Hopefully, there are different approaches and quite a large selection of products that can help here.

Executable wrappers can be used when the application to distribute does not need any configuration of the target system to operate correctly. It is also possible with this approach to optionally include a JRE for those users who do not want to bother about Java. The drawback is that the application has to be compressed for easy distribution, which can be confusing for some users.

Installers generators let developers create installer programs for their applications. As most users expect to find a single executable file they can run in the standard way of their operating system, allowing them to setup the application through a graphical wizard, this seems to be the natural approach to opt for. The drawbacks are specific to the generator chosen.

Finally, Java Deployment can become the best approach in the future, at least for users who have an Internet access and Java already installed on their computer. These may be seen as the drawbacks, the advantage being that the application can be automatically and transparently updated when new versions are released.

There is a wide range of solutions suited to almost every needs. The open source community is very active, which means that the solutions investigated here are always evolving. Future studies can include topics not covered here in details, e.g. Package Management Systems, solutions specific for Mac OS, support for third-party libraries such as other database systems (Microsoft Access, MySQL, etc.).

# 7 References

## *7.1 Executable wrappers*


- Jsmooth, http://jsmooth.sourceforge.net
- Launch4j, http://launch4j.sourceforge.net
- One-JAR, http://one-jar.sourceforge.net
- Simplify your application delivery with One-JAR, http://www-128.ibm.com/developerworks/java/library/j-onejar/


14 Will Iverson, strategic partner manager for Symantec's Internet Tools Division.

## 7.2 Installers generators

- Desktop Authority MSI Studio (http://www.scriptlogic.com)
- FLEXnet InstallShield & InstallAnywhere (http://www.macrovision.com)
- Inno Setup, http://www.jrsoftware.org/isinfo.php
- InstallAnywhere Now, http://goldengate.zerog.com/releases/now
- InstallBuilder, http://www.bitrock.com/products_installbuilder_overview.html
- IzPack, http://www.izforge.com/izpack
    - Building Native Windows Installers with IzPack Native Launcher, http://www.javalobby.org/articles/izpack/
- Nullsoft Scriptable Install System, http://nsis.sourceforge.net/Main_Page
- SetupBuilder (http://www.lindersoft.com)
- Spoon Installer, http://sourceforge.net/projects/spoon-installer
- VAInstall, http://vainstall.sourceforge.net
- Wise Installation (http://www.wise.com)
- WiX, http://wix.sourceforge.net

## 7.3 Java Deployment

- Lesson: Packaging Programs in JAR Files (Java tutorial on deployment), http://java.sun.com/docs/books/tutorial/deployment/jar/index.html

### 7.3.1 Applets

- Java Plugin Guide, http://java.sun.com/j2se/1.5.0/docs/guide/plugin/developer_guide/contents.html
- Lesson: Applets (Java tutorial on deployment), http://java.sun.com/docs/books/tutorial/deployment/applet/index.html

### 7.3.2 Web Start

- Java Web Start Guide, http://java.sun.com/j2se/1.5.0/docs/guide/javaws/developersguide/contents.html
- Java Web Start and Code Signing, http://www.dallaway.com/acad/webstart/index.html
- JNLP specification, http://jcp.org/en/jsr/detail?id=56